



الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يُونُسُ بَرَسِيَّتِي إِسْلَامُ إِنْتَارَا بَغْسِيَا مِلِّيْسِيَا
Garden of Knowledge and Virtue

MCTA 3203

MECHATRONICS SYSTEM INTEGRATION

SEMESTER 1, 25/26

HARDWARE COMPONENTS INTEGRATION AND CODING

SECTION 2

GROUP 19

LECTURER : DR WAHJU SEDIONO

DATE OF EXPERIMENT : 28 DECEMBER 2025

DATE OF SUBMISSION : 2 JANUARY 2026

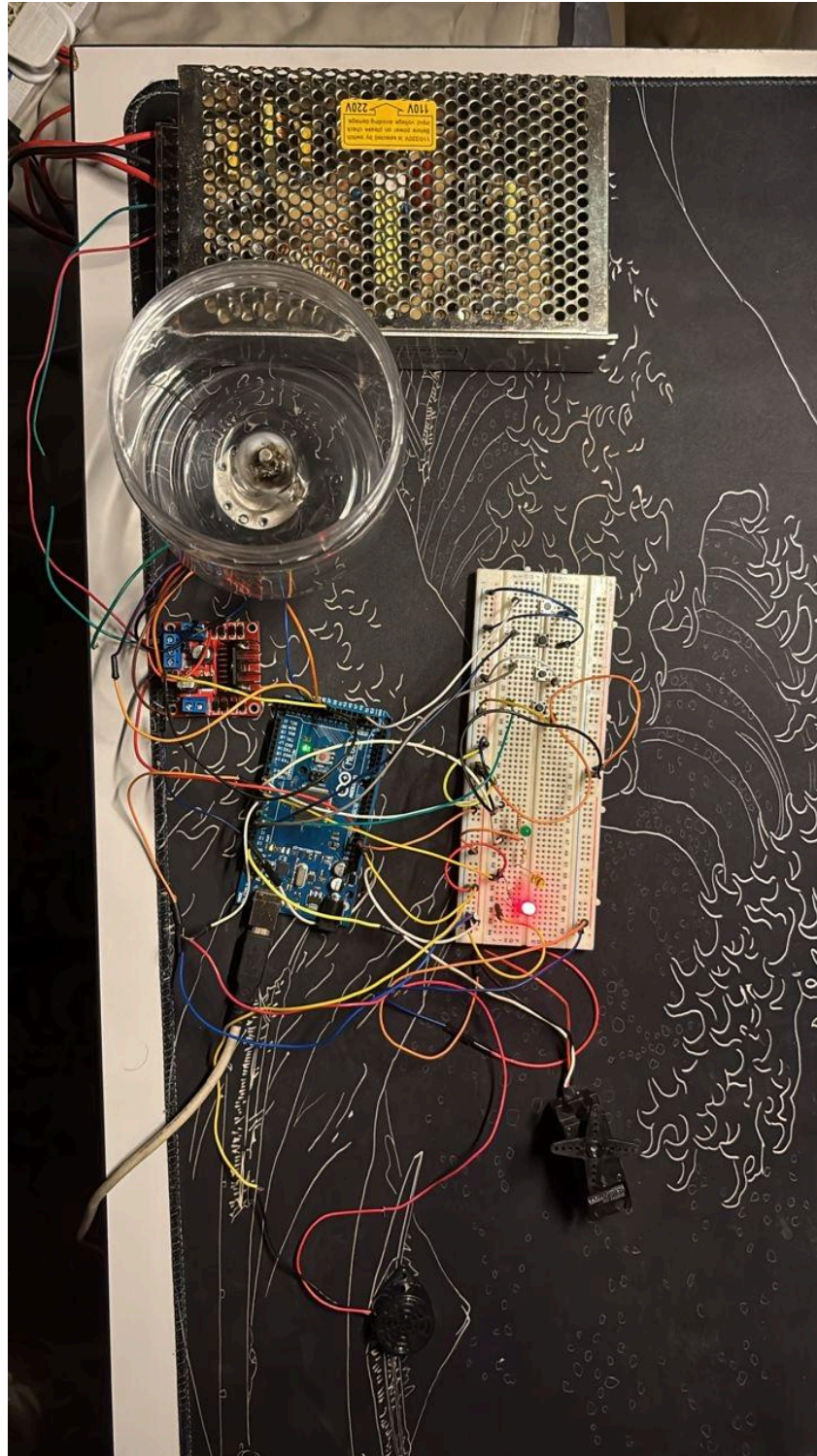
PREPARED BY :

NAME	MATRIC NUMBER
MUHAMMAD IMRAN BIN VEDDIN	2316409
ALYA KHAIRAH BINTI KHAIRUL JAMIL	2317100
MUHAMMAD IRFAN RUSHDI BIN ASRI	2319011

TABLE OF CONTENTS

1.0 HARDWARE COMPONENTS INTEGRATION	3
2.0 CODING	4

1.0 HARDWARE COMPONENTS INTEGRATION



2.0 CODING

```
#include <Servo.h> // MUST BE AT TOP

// ----- BUTTONS -----
#define WASH_BTN 8
#define RINSE_BTN 9
#define SPIN_BTN 10
#define ESTOP_BTN 53

// ----- LEDS -----
#define GREEN_LED 2
#define YELLOW_LED 3
#define RED_LED 4

// ----- BUZZER -----
#define BUZZER 6

// ----- SERVO -----
#define SERVO_PIN 13
#define SERVO_OPEN 90
#define SERVO_CLOSE 0

// ----- L298N MOTOR -----
#define ENA 39 // PWM
#define IN1 37
#define IN2 35

Servo drainServo;
bool emergencyActive = false;

// ----- MOTOR CONTROL -----
void motorCW(int speedPWM) { // Clockwise
    digitalWrite(IN1, HIGH);
```

```

digitalWrite(IN2, LOW);
analogWrite(ENA, speedPWM);
}

void motorCCW(int speedPWM) {    // Counter-clockwise
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    analogWrite(ENA, speedPWM);
}

void motorOff() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    analogWrite(ENA, 0);
}

// ----- BUZZER -----
void beepOnce() {
    digitalWrite(BUZZER, HIGH);
    delay(100);
    digitalWrite(BUZZER, LOW);
}

void beepEnd() {
    for (int i = 0; i < 3; i++) {
        digitalWrite(BUZZER, HIGH);
        delay(150);
        digitalWrite(BUZZER, LOW);
        delay(150);
    }
}

// ----- EMERGENCY CHECK -----
void emergencyCheck() {

```

```

if (digitalRead(ESTOP_BTN) == LOW) {
    emergencyActive = true;
    motorOff();
    drainServo.write(SERVO_CLOSE);
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(RED_LED, HIGH);
}
}

// ----- SAFE DELAY -----
void safeDelay(unsigned long duration) {
    unsigned long start = millis();
    while (millis() - start < duration) {
        emergencyCheck();
        if (emergencyActive) return;
    }
}

// ----- WASHING AGITATION -----
void washAgitate(unsigned long totalTime, int speedPWM) {
    unsigned long startTime = millis();

    while (millis() - startTime < totalTime) {
        emergencyCheck();
        if (emergencyActive) return;

        motorCW(speedPWM);
        safeDelay(2000);

        motorOff();
        safeDelay(500);

        motorCCW(speedPWM);

```

```

    safeDelay(2000);

    motorOff();
    safeDelay(500);
}
}

// ----- SPIN MODE -----
void spinMode(unsigned long totalTime, int speedPWM) {
    motorCW(speedPWM);
    safeDelay(totalTime);
    motorOff();
}

// ----- MAIN PROCESS -----
void runProcess(const char* processName) {
    if (emergencyActive) return;

    beepOnce();

    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(RED_LED, LOW);

    // ----- PROCESS TYPES -----
    if (strcmp(processName, "WASH") == 0) {
        washAgitate(10000, 140); // 10s wash agitation
    }

    if (strcmp(processName, "RINSE") == 0) {
        washAgitate(8000, 160); // Faster, shorter
    }

    if (strcmp(processName, "SPIN") == 0) {

```

```

    spinMode(6000, 220);    // High speed spin
}

if (emergencyActive) return;

// ----- DRAIN -----
motorOff();
digitalWrite(GREEN_LED, LOW);
digitalWrite(YELLOW_LED, HIGH);

drainServo.write(SERVO_OPEN);
safeDelay(5000);
drainServo.write(SERVO_CLOSE);

if (emergencyActive) return;

// ----- END -----
motorOff();
digitalWrite(YELLOW_LED, LOW);
digitalWrite(RED_LED, HIGH);
beepEnd();

// Wait until button released
while (
    digitalRead(WASH_BTN) == LOW ||
    digitalRead(RINSE_BTN) == LOW ||
    digitalRead(SPIN_BTN) == LOW
);
}

// ----- SETUP -----
void setup() {
    pinMode(WASH_BTN, INPUT_PULLUP);
    pinMode(RINSE_BTN, INPUT_PULLUP);

```



```
pinMode(SPIN_BTN, INPUT_PULLUP);
pinMode(ESTOP_BTN, INPUT_PULLUP);
```

```
pinMode(GREEN_LED, OUTPUT);
pinMode(YELLOW_LED, OUTPUT);
pinMode(RED_LED, OUTPUT);
pinMode(BUZZER, OUTPUT);
```

```
pinMode(ENA, OUTPUT);
pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
```

```
motorOff();
```

```
drainServo.attach(SERVO_PIN);
drainServo.write(SERVO_CLOSE);
```

```
digitalWrite(RED_LED, HIGH); // Idle
}
```

```
// ----- LOOP -----
```

```
void loop() {
    emergencyCheck();
```

```
    // Reset emergency when released
```

```
    if (emergencyActive && digitalRead(ESTOP_BTN) == HIGH) {
        emergencyActive = false;
        digitalWrite(RED_LED, HIGH);
    }
```

```
    if (!emergencyActive) {
        if (digitalRead(WASH_BTN) == LOW) {
            runProcess("WASH");
        }
    }
```

```
if (digitalRead(RINSE_BTN) == LOW) {  
    runProcess("RINSE");  
}  
if (digitalRead(SPIN_BTN) == LOW) {  
    runProcess("SPIN");  
}  
}  
}
```